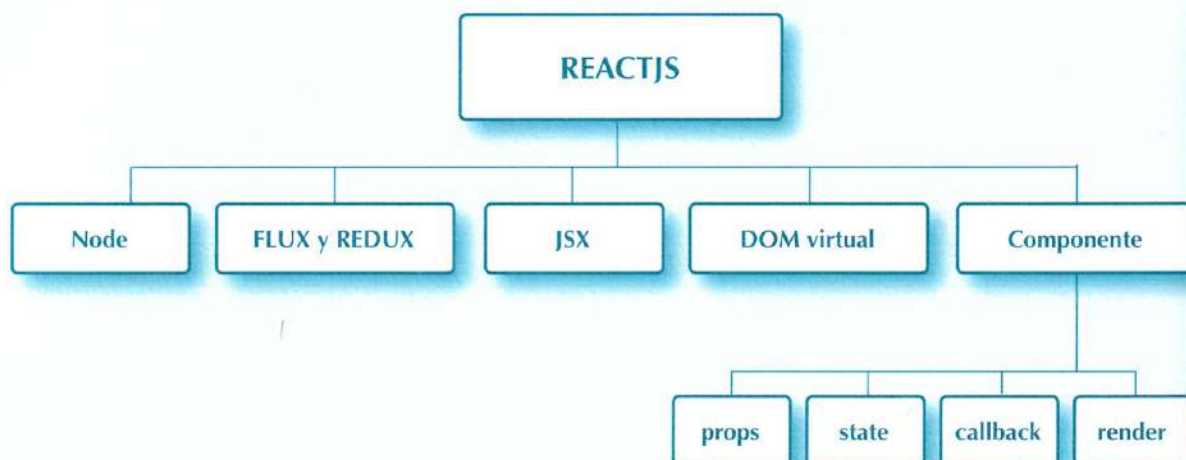


ReactJS: un framework de JavaScript

Objetivos

- ✓ Aprender la programación orientada a componentes.
- ✓ Comprender el concepto de *componente* con su state y sus props.
- ✓ Trabajar otro lenguaje de programación como es JSX.
- ✓ Conocer el concepto de *callback* o *feedback* de los componentes hijos hacia los padres.
- ✓ Utilizar y conocer el concepto de *DOM virtual*.
- ✓ Entender el framework o librería ReactJS, que es actualmente el más utilizado en la programación front-end.
- ✓ Crear una aplicación sencilla con ReactJS en la que poder tener una visión lo más amplia de la utilización de este framework.

Mapa conceptual



Glosario

Callback. Llamada que hace un componente hijo a un método de un componente padre. Es la manera que tiene un componente hijo de comunicarse con su componente padre.

Componente. La programación orientada a componentes es un tipo de programación en la que los sistemas se descomponen en componentes funcionales o presentacionales (relativos a la interfaz). El nivel de abstracción es más alto que el de los objetos.

Elemento. Pequeña porción de código en una aplicación React.

JSX. Extensión de JavaScript utilizada para programar componentes React.

Props. Sistema de comunicación entre componentes que utiliza React. Los componentes padre envían información a los componentes hijo a través de las props.

Renderizar. Proceso por el cual el componente se dibuja en el interfaz (navegador).

State. Estado de un componente donde se almacena la información relativa a este. Al igual que los objetos tienen atributos, un componente tiene estado.

Virtual DOM. DOM virtual utilizado por React. Al utilizar un DOM virtual los cambios en la interfaz son más ágiles y rápidos.

6.1. Introducción a React

ReactJS es un framework que permite a los programadores realizar aplicaciones web utilizando programación declarativa. Las aplicaciones React están formadas por componentes y React es capaz de renderizar dichos componentes de una manera sencilla, rápida y eficiente siempre que alguno de ellos sufra una modificación.

Como se ha explicado anteriormente, React utiliza programación orientada a componentes (que no objetos). Los componentes gestionan su propio estado y, agrupando una serie de componentes, los programadores son capaces de crear una interfaz de usuario.

Al ser los componentes elementos autónomos, su reutilización es muy alta y sencilla.

FUNDAMENTAL

Node.js

Para trabajar con React, es preciso instalar en el equipo Node.js. La instalación de Node es sumamente sencilla. En Ubuntu, bastaría con ejecutar los siguientes comandos:

1. `$ sudo apt install nodejs`
2. `$ sudo apt install npm`

El primero para instalar Node y el segundo para instalar el gestor de paquetes de node (npm - node package manager).

ReactJS utiliza una sintaxis parecida al XML llamada JSX. Lo normal en ReactJS es programar en JSX, pero no es absolutamente necesario.

A lo largo de este capítulo, se aprenderá a utilizar ReactJS creando un sudoku interactivo, para lo cual va a utilizarse el código del ejercicio resuelto del sudoku del capítulo 3 de este libro.

Recursos web

www

En el primer código QR, encontrarás un enlace de Wikipedia con más información sobre React. A través del segundo código QR, podrás acceder a un enlace de Netguru para obtener más información sobre programación declarativa e imperativa.

1.



2.



React fue diseñado por Jordan Walke y su primera versión se utilizó en las newsfeed de Facebook en el 2011. Se hizo open-source en la conferencia JSConf en los Estados Unidos en mayo del 2013. Este framework se diseñó para que fuese rápido, simple y escalable.

Por lo tanto, ReactJS es:

- Una *librería JavaScript* para crear interfaces de usuario.
- Utiliza *programación declarativa* y no imperativa como puede ser Java, por ejemplo.
- Utiliza *programación component-based* (basada en componentes).
- *Technology stack agnostic*. Esto quiere decir que interactúa bien con cualquier arquitectura. No obliga a utilizar una arquitectura determinada.

6.1.1. Creación de la primera aplicación React

En este apartado, se va a crear la primera aplicación React. En este caso, se va a emplear la utilidad `create-react-app`, que crea el esqueleto de una aplicación React. Este esqueleto podrá ser utilizado para modificarlo y dar forma a la aplicación que se quiera desarrollar.

© FUNDAMENTAL

También se puede instalar YARN

Yarn es una herramienta para gestionar paquetes de aplicaciones Node.js. Es fiable, rápido y seguro y, por lo tanto, es muy utilizado en Node.js. Se puede instalar en Ubuntu de una forma sencilla y rápida.

Existen varios métodos para instalar Yarn, pero uno muy directo sería instalarlo utilizando NPM, para ello basta con ejecutar el siguiente comando (para instalar Yarn de forma global y no solamente para el proyecto actual, hay que utilizar el parámetro `-g`):

```
sudo npm install yarn -g
```



```
morenoperezjc@mini: ~/react
Archivo Editar Ver Buscar Terminal Ayuda
morenoperezjc@mini:~/react$ sudo npm install yarn -g
/usr/local/bin/yarn -> /usr/local/lib/node_modules/yarn/bin/yarn.js
/usr/local/bin/yarnpkg -> /usr/local/lib/node_modules/yarn/bin/yarn.js
/usr/local/lib
└─ yarn@1.9.4
morenoperezjc@mini:~/react$
```

Figura 6.1
Instalación de Yarn.

Una vez ejecutado este comando, Yarn estará instalado correctamente.

El primer paso es instalar la aplicación `create-react-app` mediante NPM (Node Package Manager).

En Ubuntu, se puede instalar mediante los siguientes comandos:

```
sudo npm install -g create-react-app
```

o

```
sudo npm install -g create-react-app@1.5.2
```

El segundo comando sirve para instalar una versión concreta de create-react-app.

Para crear la primera aplicación, bastará con ejecutar create-react-app. En Ubuntu, se ejecutaría con el siguiente comando:

```
create-react-app sudoku
```

Donde sudoku es el nombre de la aplicación que ha de crearse.

El comando anterior creará un esqueleto de una aplicación básica en React con el contenido que se indica en la figura 6.3.

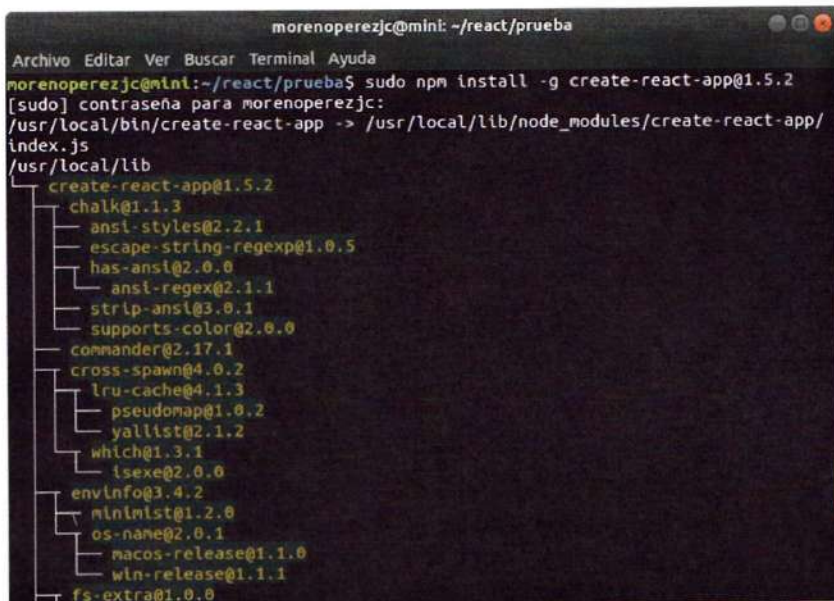


Figura 6.2
Instalando create-react-app.



Figura 6.3
Contenido de la primera aplicación React (Atom IDE).

6.1.2. Ejecución de la aplicación React

El siguiente paso es ejecutar la aplicación. En Ubuntu, para ejecutar una aplicación, basta con estar en el directorio donde se ubique esta y ejecutar el comando:

```
npm start o yarn start
```

Una vez ejecutado el comando, se abrirá el navegador por defecto y se ejecutará la aplicación.

6.2. Análisis del contenido de la primera aplicación

En este apartado, se analizará el contenido (ficheros) de la primera aplicación con React creada en el apartado 6.1.1. Los archivos más importantes de la aplicación son los siguientes:

- El archivo HTML *public/index.html*.
- El archivo de script *src/App.js*.
- El fichero de script *src/index.js*.

RECUERDA

- ✓ Los componentes React comienzan con letra mayúscula y el resto del nombre estará escrito en minúsculas.
Las etiquetas HTML deberán estar en minúsculas en el código JSX.

6.2.1. El fichero *index.html*

La siguiente línea es la más interesante de este fichero:

```
<div id="root"></div>
```

Como se puede observar, hay un elemento de tipo `<div>` con `id="root"`. Justo en este elemento, se renderizará la aplicación React.

6.2.2. El fichero *src/App.js*

Este archivo es donde residirá la lógica de la aplicación. Interpretando el contenido, se puede adivinar lo que se visualizará en el navegador.

Es importante destacar que *App* extiende *Component*, luego la aplicación tendrá un componente principal React llamado *App*.



SABÍAS QUE...

El método `render()` en un componente React es el encargado de mostrar el componente en la interfaz web.

La estructura visual de la aplicación está en el método `render()`, pues es el que devuelve el contenido HTML y el que dibuja el componente de la aplicación. Es uno de los más importantes, puesto que va a definir o mostrar la funcionalidad del componente.

El componente *App* se exporta para poder ser utilizado después por otras partes de la aplicación:

```
export default App;
```


6.2.3. El fichero index.js

En este fichero, se importan los distintos elementos de los que consta la aplicación y que son imprescindibles para ella. Crea el enlace entre la aplicación y el elemento root definido en index.html mediante la siguiente línea:

```
ReactDOM.render(<App />, document.getElementById('root'));
```

Esta orden renderiza la aplicación en el elemento de index.html con el “id=“root”.

TOMA NOTA



Las aplicaciones React están creadas basándose en pequeñas porciones de código. Por ejemplo:

```
const element = <h1 className="App-title">Welcome to  
React</h1>;
```

En estos elementos, se utiliza `className` en vez de `class` para no confundirse con las clases. Como se puede observar, para que un elemento sea eficiente, los objetos JavaScript que se utilicen tienen que ser sencillos y fáciles de crear.

Hay que tener en cuenta que los `components` o componentes están generados con `elements` o elementos.

6.3. Introducción a JSX

JSX es una extensión de JavaScript. Aunque JSX parece un lenguaje de plantillas, se puede utilizar JavaScript sin problemas para darle una mayor potencia.

JSX genera elementos React (muchos elementos HTML tienen una correspondencia con un elemento React).

Entre las ventajas que tiene JSX es que está íntimamente ligado con el manejo de eventos, con cómo el estado de los componentes cambia durante su vida y cómo se formatean los datos para ser mostrados en la interfaz.

Los componentes de React contienen la lógica y la parte de interfaz en el mismo componente.

Es importante tener en cuenta que React no necesita utilizar JSX, pero los desarrolladores lo utilizan porque encuentran sumamente útil tener el HTML dentro del código JavaScript, puesto que está íntimamente ligado. Además, permite a React ser más útil a la hora de depurar y mostrar los mensajes de advertencia o error.

En la figura 6.4, se puede observar que los dos códigos de los elementos de la parte superior son similares para React. El código de la parte superior izquierda es JSX, mientras que el de la parte superior derecha es código JavaScript haciendo una llamada a un método de React.

Ambos códigos generarán el elemento que se describe en la parte baja de la figura.

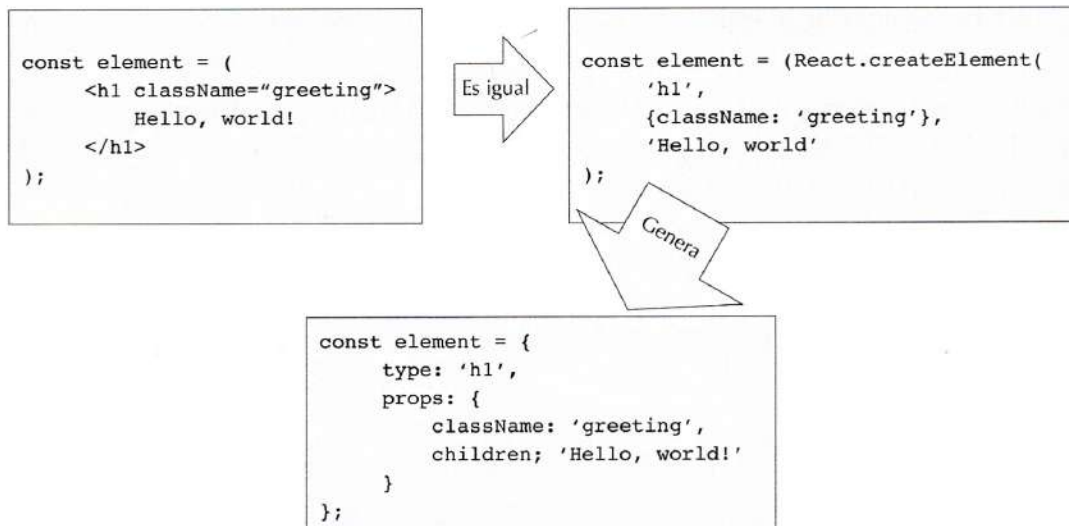


Figura 6.4
JavaScript frente a ES6.

www

Recurso web

Para más información sobre JSX, puedes consultar la página web de React a través del siguiente código QR:



6.4. Componentes React

Cuando se desarrolla una aplicación React, hay que tener en cuenta lo siguiente sobre los componentes:

- Las interfaces de usuario se compondrán de componentes que son independientes unos de otros.
- Los componentes son piezas reutilizables, como las piezas de LEGO.
- Un componente devolverá una serie de elementos que aparecerán en la ventana del navegador.
- En React, se pueden definir diferentes clases o tipos de componentes.



TOMA NOTA

- Los componentes realizados por el programador deben tener la primera letra en mayúsculas obligatoriamente. Esos componentes serán compilados con el método `React.createElement(.....)`.
- Las etiquetas que empiezan por minúsculas se tratarán como etiquetas del DOM.
- También hay que tener en cuenta que existen componentes predefinidos.

La aplicación que se va a desarrollar en posteriores apartados de este capítulo será un sudoku de 81 casillas y tendrá la jerarquía de componentes que se muestra en la figura 6.5.

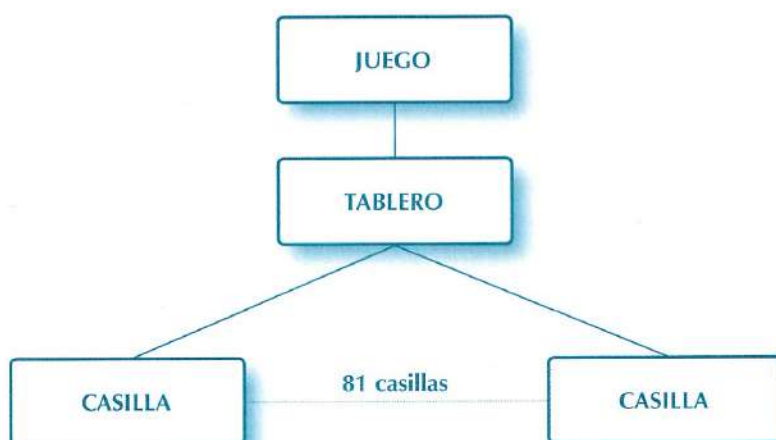


Figura 6.5
Jerarquía de componentes de la aplicación para crear un juego de sudoku.

Existirá un componente principal llamado *Juego*, que, a su vez, tendrá el componente *Tablero*. El componente *Tablero* será el más complejo, puesto que almacenará la mayor parte de la lógica de la aplicación.

El componente *Tablero* tiene a su cargo 81 componentes hijo (81 casillas, es decir, 9×9 casillas).

6.4.1. Creación del componente principal: el tablero del sudoku

Partiendo de la primera aplicación React creada con create-react-app, se eliminarán todos los archivos y se cambiará el código del fichero index.js por el que se proporciona a continuación:

```

import React from 'react';
import ReactDOM from 'react-dom';

class Tablero extends React.Component {

  render() {

    return (
      <div>
        <div className="tablero-fila">
          X X X X X X X X X
        </div>
        <div className="tablero-fila">
          X X X X X X X X X
        </div>
      </div>
    );
  }
}

```

```

        </div>
        <div className="tablero-fila">
            x x x x x x x x x
        </div>
        <div className="tablero-fila">
            x x x x x x x x x
        </div>
        <div className="tablero-fila">
            x x x x x x x x x
        </div>
        <div className="tablero-fila">
            x x x x x x x x x
        </div>
        <div className="tablero-fila">
            x x x x x x x x x
        </div>
        <div className="tablero-fila">
            x x x x x x x x x
        </div>
        <div className="tablero-fila">
            x x x x x x x x x
        </div>
    </div>
    );
}
}

class Juego extends React.Component {
  render() {
    return (
      <div className="juego">
        <div className="juego-tablero">
          <Tablero />
        </div>
        <div className="juego-info">
        </div>
      </div>
    );
  }
}

// =====

ReactDOM.render(
  <Juego />,
  document.getElementById('root')
);

```


En el código anterior, se puede ver que, al final de este, se renderiza el componente Juego, el cual es el padre del componente Tablero, dado que el método `render()` de Juego crea el componente Tablero mediante la siguiente llamada:

```
<Tablero />
```

Tablero, a su vez, va a dibujar una serie de filas, las cuales, en este momento, estarán formadas cada una por nueve equis (X). Posteriormente, se crearán las casillas del sudoku.

El aspecto de la aplicación React al ejecutar `npm start` o `yarn start` será el que se muestra en la figura 6.6.



Figura 6.6
Aspecto de la aplicación React.



Actividad propuesta 6.1

Comprueba que funciona el código expuesto en este apartado en su instalación Node. Intenta añadir alguna variante al juego propuesto para así conocer mejor cómo funciona ReactJS.

6.5. Tipos de componentes. Contenedores y presentacionales

Los componentes en React se clasifican dependiendo de su uso (de cómo se utilizan). Algunas clasificaciones son las siguientes:

- Presentación/presentacionales o contenedores (Presentational frente a Container).
- Flacos y gordos (Skinny frente a Fat).
- Tontos y listos (Dumb frente a Smart).
- Sin estado y con estado (Stateless frente a Stateful).

A continuación, se describirán los dos primeros grupos, puesto que son los más utilizados:

1. Los componentes *presentacionales*:

- Están orientados a renderizar la vista. Se preocupan de que tengan un aspecto visual agradable. Utilizan estilos predefinidos para mejorar la presentación.

- Renderizan la vista dependiendo de los datos que se les proporcionen.
- No hace falta que gestionen su propio estado. No hace falta tampoco que manejen datos, solamente los necesarios para realizar la presentación de la interfaz.

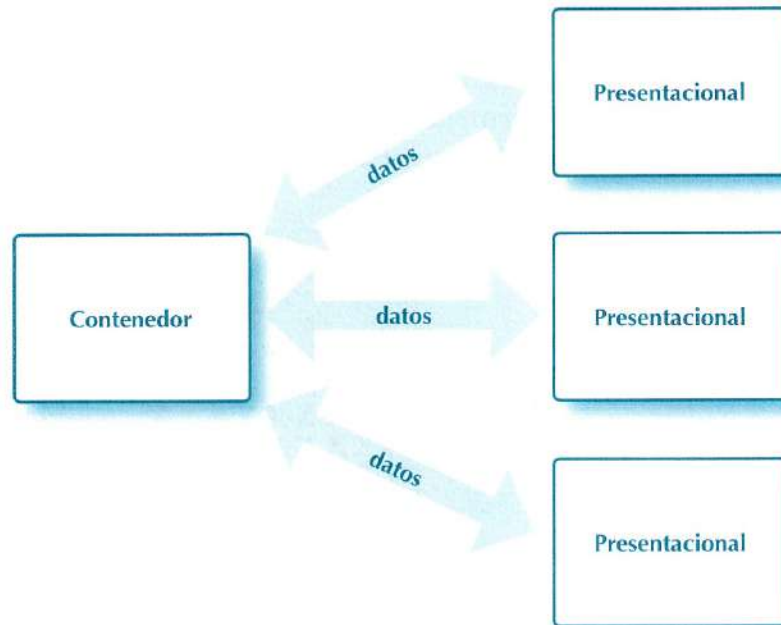


Figura 6.7

Componentes contenedores y presentacionales.

2. Los componentes *contenedores*:

- Son los responsables de que la aplicación funcione correctamente. Pueden dedicarse a realizar consultas a bases de datos, actualizar el estado de los componentes, etcétera.
- Realizan llamadas a los componentes presentacionales para mostrar la interfaz al usuario. También les envían datos a estos componentes para que los muestren en el navegador.
- Generalmente, estos componentes presentacionales están embebidos en forma de `div`s en estos contenedores.
- Son los responsables de comunicarse con las bases de datos u otros orígenes de datos.

Por lo tanto:

- Los componentes Tablero y Juego serán un componente contenedor.
- Los componentes Casilla serán componentes presentacionales.

6.6. Los componentes. State y props de un componente

Como ya se ha indicado, `state` representará el estado del componente y las `props` (o propiedades) son los parámetros que le pasa un componente padre a su componente subordinado o hijo.



Flux y Redux son las tecnologías que utiliza Facebook para realizar sus interfaces con ReactJS. ¿Qué diferencias hay entre FLUX y REDUX?

- Flux es un patrón de diseño, mientras que Redux –que es más complejo– es una librería. En React, se comenzó utilizando Flux, pero ahora se utiliza Redux porque tiene más ventajas que el primero.
- *Flux* es el nombre que se le da al patrón de diseño observer (observador), el cual se modificó levemente para utilizarlo en React.
- Tanto Flux como Redux tienen acciones, que serían los eventos o lo que desencadenan los eventos. De todas formas, cuando se utiliza un middleware como Redux, las acciones también pueden ser funciones o promesas (promises).
- En Flux, el almacenamiento de los componentes puede tener varias ubicaciones, mientras que, en Redux, el almacenamiento está centralizado. Esta es una de las claves por las que, al trabajar con React, se utiliza Redux en vez de Flux, por la centralización de la información de la aplicación que hace Redux.

En definitiva, ambas se basan en una aplicación que tiene un estado inicial y va a ir modificándose según los eventos acaecidos.

6.6.1. El state

Sobre el state, hay que tener en cuenta los siguientes conceptos:

- Cada componente tiene su información local registrada en su estado (state). Esa información es controlada solamente por el componente, ningún otro componente puede acceder directamente a ella.
- Ese estado se puede pasar a los componentes hijos o subordinados como props. De esa forma, un ancestro (ancestor) puede pasar información a toda su descendencia (descendant).
- Solamente los class components pueden tener estado local (state).

Es importante conocer que el estado se declara en el constructor (dentro de él) de la siguiente manera:

```
class Juego extends Component {
  constructor(props) {
    super(props);
    this.state = {
      cuadros: Array(81).fill(null),
      cuadrosShown: Array(81).fill(null),
      capado: Array(81).fill(false),
      fallos: 0,
      acertados: 0,
    }
  }
}
```

```

        jugando: "jugando...",
      };
    }
    . . .
  }

```

RECUERDA

- ✓ El estado (state) solamente puede ser modificado utilizando el método `setState()`. Un ejemplo de este método sería el siguiente:

```

incFallo(){
  const
  fallos = this.state.fallos;
  this.setState({fallos:fallos+1});
}

```

La siguiente sentencia no funcionará:

```

this.state.fallos += 1;

```

6.6.2. Las props

JSX pasa información de un componente a otro mediante props, las cuales son tratadas como un objeto individual.

Puede accederse a esta información dentro del componente mediante sus props y se puede pasar más de una propiedad (prop).

www

Recurso web

A través del código QR adjunto, accederás a un enlace de React donde se muestra cómo se renderizan los componentes y cómo se utilizan correctamente las props



Un componente no puede modificar las props. Si se quiere modificar la información que viene vía props, lo más lógico es copiar el contenido en el state del componente y luego modificar lo que haga falta.

Véanse algunos ejemplos de props pasadas a componentes:

```

<Casilla value={this.state.valor} posicion={i} />

```


En el ejemplo anterior, se puede observar que el componente Casilla recibe vía props dos parámetros: el primero llamado *value* y el segundo llamado *posición*.

El componente Casilla podrá acceder a las props mediante las variables:

```
this.props.value y this.props.posicion
```



Actividad propuesta 6.2

Instala las developer tools. Necesitas herramientas para conocer el state y las props de los componentes o mucha otra información interesante de tus aplicaciones React. En el siguiente enlace, hay extensiones para Firefox y para Chrome, las cuales van a permitir depurar y mejorar las aplicaciones de una forma sencilla y rápida: <https://github.com/facebook/react-devtools>. Instálalas en el navegador y comprueba cómo funcionan.

6.7. El DOM virtual de React

Como ya se ha indicado, el DOM (Document Object Model) es un objeto del navegador. Al igual que el navegador, React utiliza un objeto que mantiene un DOM virtual, el cual es una representación ligera del DOM del navegador.

React tiene mapeados los objetos del DOM a objetos virtuales propios. El DOM virtual se almacena como un árbol en la memoria, donde cada nodo será un objeto que representará a un elemento.

Al residir en la memoria, este DOM virtual es muy rápido y sus modificaciones son mucho más ágiles que las del DOM del navegador.

6.7.1. Renderización y cambios

React tiene un algoritmo que detecta los nodos que cambian y se encarga de re-renderizarlos. Además, si cambia un nodo y se renderiza, los nodos descendientes tienen que renderizarse de nuevo, puesto que un ancestro ha cambiado y dichos cambios podrían afectarles.

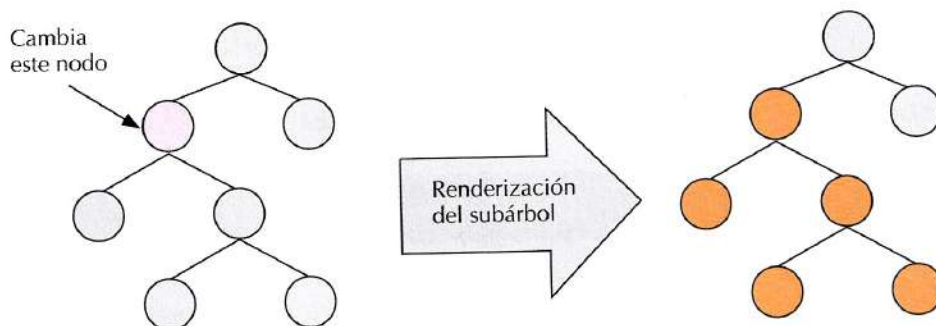


Figura 6.8
Cambios ante una renderización.

Por ejemplo, en la figura 6.8, se puede observar que React ha detectado que un nodo del DOM virtual ha cambiado. React realizará un recorrido hacia abajo renderizando todos los objetos del DOM virtual descendientes del nodo modificado.

6.8. Los callbacks

Imagínese un caso frecuente en el que un componente necesita enviar una información a otro componente subordinado y necesita algún feedback (cierta información de vuelta).

Póngase un caso concreto, un componente ficha de paciente que tiene otros componentes subordinados como diagnósticos del paciente, medicamentos tomados por el paciente, vacunas, etc. El componente ficha necesitará tener constancia de cualquier cambio en los componentes subordinados como la introducción de un nuevo medicamento por ejemplo. El componente ficha será el encargado de:

- Actualizar los cambios o la ficha completa del paciente en la base de datos, o es el responsable de llamar al componente que haga este trabajo.
- Enviar a los componentes subordinados la información (diagnósticos, medicamentos, vacunas, etc.) para que lo visualicen.

Este envío de vuelta de información se denomina *callback* y es una función que se pasa al componente subordinado (hijo) para que la ejecute con sus datos. Esta función modificará el state del componente padre (figura 6.9).

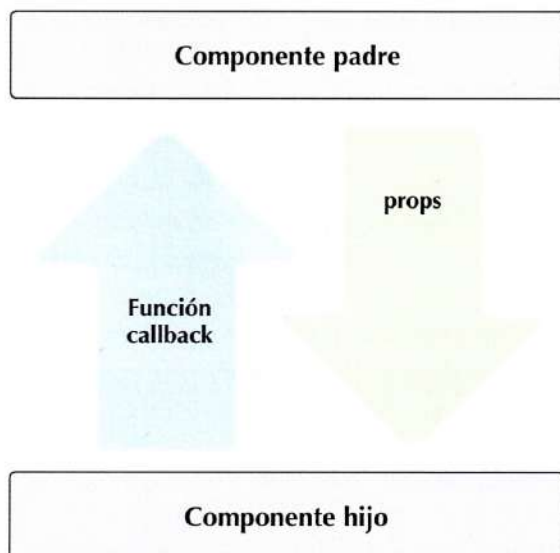


Figura 6.9

Comunicación entre componentes padre y componentes hijo.

www

Recurso web

Para más información acerca de cómo funcionan los callbacks, puedes consultar un artículo de Jason Arnold mediante el código QR adjunto.



6.9. Importar funciones de un JavaScript externo

En el capítulo 3 de este libro, ya se han utilizado funciones para crear un sudoku aleatorio a partir de un sudoku resuelto. En esta aplicación React, se va a hacer uso del código ya existente. Este código se va a ubicar en un fichero llamado *sudoku.js* que va a ser utilizado por el fichero principal de la aplicación (*index.js*).

En primer lugar, se tendrán que importar tres funciones del fichero *index.js* y, para ello, habrá que hacer las siguientes modificaciones al fichero *index.js*:

```
import { aleatorio, getSudoku, generaSudoku } from './sudoku.js';
```

Escribiendo en la cabecera la línea anterior, se importarán las tres funciones y podrán ser utilizadas en el código JSX.

El fichero *sudoku.js* quedará como sigue:

```
var modificado=[
    2,1,5,9,3,8,4,6,7,
    7,8,6,1,2,4,3,5,9,
    9,3,4,6,5,7,2,8,1,
    8,6,9,5,4,2,1,7,3,
    1,4,3,7,8,6,5,9,2,
    5,2,7,3,9,1,8,4,6,
    6,7,2,4,1,5,9,3,8,
    4,9,8,2,6,3,7,1,5,
    3,5,1,8,7,9,6,2,4];

export function getSudoku(pos){
    return modificado[pos];
}

export function generaSudoku(){
    cambiaColumnas();
    cambiaFilas();
    cambiaColumnas();
    cambiaFilas();
}

export function aleatorio(min,max){
    var horquilla=max-min;
    var aleatorio = Math.round(Math.random()*horquilla)+min;
    return aleatorio;
}

function cambiaColumnas(){
    var pos=0,i,aux;
    var columnaA=aleatorio(0,2)+pos;
    var columnaB=aleatorio(0,2)+pos;
```

```
for (i=0;i<9;i++){
    aux = modificado[columnaA];
    modificado[columnaA]=modificado[columnaB];
    modificado[columnaB]=aux;
    columnaA+=9;
    columnaB+=9;
}

pos=3;
columnaA=aleatorio(0,2)+pos;
columnaB=aleatorio(0,2)+pos;

for (i=0;i<9;i++){
    aux = modificado[columnaA];
    modificado[columnaA]=modificado[columnaB];
    modificado[columnaB]=aux;
    columnaA+=9;
    columnaB+=9;
}

pos=6;
columnaA=aleatorio(0,2)+pos;
columnaB=aleatorio(0,2)+pos;

for (i=0;i<9;i++){
    aux = modificado[columnaA];
    modificado[columnaA]=modificado[columnaB];
    modificado[columnaB]=aux;
    columnaA+=9;
    columnaB+=9;
}
}

function cambiaFilas(){
    var pos=0,i,aux;
    var filaA=9*aleatorio(0,2)+pos;
    var filaB=9*aleatorio(0,2)+pos;

    for (i=0;i<9;i++){
        aux = modificado[filaA];
        modificado[filaA]=modificado[filaB];
        modificado[filaB]=aux;
        filaA+=1;
        filaB+=1;
    }

    pos=27;
    filaA=9*aleatorio(0,2)+pos;
```



```

    filaB=9*aleatorio(0,2)+pos;

    for (i=0;i<9;i++){
        aux = modificado[filaA];
        modificado[filaA]=modificado[filaB];
        modificado[filaB]=aux;
        filaA+=1;
        filaB+=1;
    }

    pos=54;
    filaA=9*aleatorio(0,2)+pos;
    filaB=9*aleatorio(0,2)+pos;

    for (i=0;i<9;i++){
        aux = modificado[filaA];
        modificado[filaA]=modificado[filaB];
        modificado[filaB]=aux;
        filaA+=1;
        filaB+=1;
    }
}

```

TOMA NOTA



Las funciones que van a exportarse tienen la palabra clave *export* antes de la palabra clave *function*.

6.10. Los componentes hijo. El componente Casilla

A continuación, se ha modificado el código del apartado 6.9 para dar cabida al componente Casilla. Dado que este es muy sencillo, se ha optado por utilizar un function component en vez de un class component.

```

import React from 'react';
import ReactDOM from 'react-dom';
import { aleatorio, getSudoku, generaSudoku } from './sudoku.js';
import './index.css';

function Casilla(props) {
    return (
        <input type="text" className="casilla" value={props.posicion}
            maxLength="1" />
    );
}

```

```
class Tablero extends React.Component {

  renderCasilla(i) {
    return (
      <Casilla posicion={i}/>
    );
  }

  render() {

    return (
      <div>
        <div className="tablero-fla">
          {this.renderCasilla(0)}
          {this.renderCasilla(1)}
          {this.renderCasilla(2)}
          {this.renderCasilla(3)}
          {this.renderCasilla(4)}
          {this.renderCasilla(5)}
          {this.renderCasilla(6)}
          {this.renderCasilla(7)}
          {this.renderCasilla(8)}
        </div>
        <div className="tablero-fla">
          {this.renderCasilla(9)}
          {this.renderCasilla(10)}
          {this.renderCasilla(11)}
          {this.renderCasilla(12)}
          {this.renderCasilla(13)}
          {this.renderCasilla(14)}
          {this.renderCasilla(15)}
          {this.renderCasilla(16)}
          {this.renderCasilla(17)}
        </div>
        <div className="tablero-fla">
          {this.renderCasilla(18)}
          {this.renderCasilla(19)}
          {this.renderCasilla(20)}
          {this.renderCasilla(21)}
          {this.renderCasilla(22)}
          {this.renderCasilla(23)}
          {this.renderCasilla(24)}
          {this.renderCasilla(25)}
          {this.renderCasilla(26)}
        </div>
        <div className="tablero-fla">
          {this.renderCasilla(27)}
          {this.renderCasilla(28)}
        </div>
      </div>
    );
  }
}
```

```
{this.renderCasilla(29)}
{this.renderCasilla(30)}
{this.renderCasilla(31)}
{this.renderCasilla(32)}
{this.renderCasilla(33)}
{this.renderCasilla(34)}
{this.renderCasilla(35)}
</div>
<div className="tablero-fila">
  {this.renderCasilla(36)}
  {this.renderCasilla(37)}
  {this.renderCasilla(38)}
  {this.renderCasilla(39)}
  {this.renderCasilla(40)}
  {this.renderCasilla(41)}
  {this.renderCasilla(42)}
  {this.renderCasilla(43)}
  {this.renderCasilla(44)}
</div>
<div className="tablero-fila">
  {this.renderCasilla(45)}
  {this.renderCasilla(46)}
  {this.renderCasilla(47)}
  {this.renderCasilla(48)}
  {this.renderCasilla(49)}
  {this.renderCasilla(50)}
  {this.renderCasilla(51)}
  {this.renderCasilla(52)}
  {this.renderCasilla(53)}
</div>
<div className="tablero-fila">
  {this.renderCasilla(54)}
  {this.renderCasilla(55)}
  {this.renderCasilla(56)}
  {this.renderCasilla(57)}
  {this.renderCasilla(58)}
  {this.renderCasilla(59)}
  {this.renderCasilla(60)}
  {this.renderCasilla(61)}
  {this.renderCasilla(62)}
</div>
<div className="tablero-fila">
  {this.renderCasilla(63)}
  {this.renderCasilla(64)}
  {this.renderCasilla(65)}
  {this.renderCasilla(66)}
  {this.renderCasilla(67)}
  {this.renderCasilla(68)}
```



```

        {this.renderCasilla(69)}
        {this.renderCasilla(70)}
        {this.renderCasilla(71)}
      </div>
      <div className="tablero-fila">
        {this.renderCasilla(72)}
        {this.renderCasilla(73)}
        {this.renderCasilla(74)}
        {this.renderCasilla(75)}
        {this.renderCasilla(76)}
        {this.renderCasilla(77)}
        {this.renderCasilla(78)}
        {this.renderCasilla(79)}
        {this.renderCasilla(80)}
      </div>
    </div>
  );
}
}

class Juego extends React.Component {
  render() {
    return (
      <div className="juego">
        <div className="juego-tablero">
          <Tablero />
        </div>
        <div className="juego-info">
        </div>
      </div>
    );
  }
}

// =====

ReactDOM.render(
  <Juego />,
  document.getElementById('root')
);

```

Para que la aplicación tenga un aspecto mejorado, se ha creado un archivo `index.css`, el cual se importa en la cabecera del código anterior. El objetivo es mostrar el cuadro del sudoku de una forma más atractiva. El código del archivo `index.css` se muestra a continuación:

```

body {
  font: 14px "Century Gothic", Futura, sans-serif;
  margin: 20px;

```

```

}

.tablero-fila:after {
  clear: both;
  content: "";
  display: table;
}

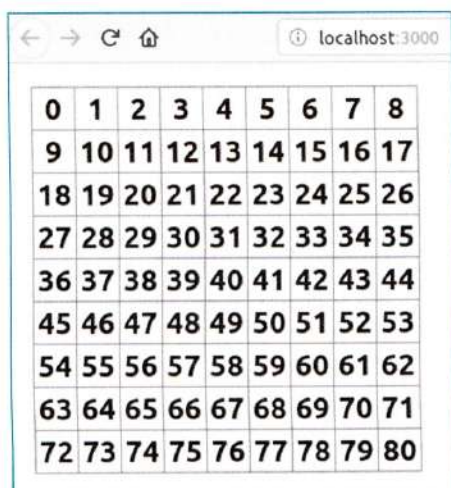
.casilla {
  background: #fff;
  border: 1px solid #999;
  float: left;
  font-size: 24px;
  font-weight: bold;
  line-height: 34px;
  height: 34px;
  margin-right: -1px;
  margin-top: -1px;
  padding: 0;
  text-align: center;
  width: 34px;
}

.casilla:focus {
  outline: none;
}

.game {
  display: flex;
  flex-direction: row;
}

```

Tras ejecutar `npm start` o `yarn start`, el resultado será igual al de la figura 6.10.



0	1	2	3	4	5	6	7	8
9	10	11	12	13	14	15	16	17
18	19	20	21	22	23	24	25	26
27	28	29	30	31	32	33	34	35
36	37	38	39	40	41	42	43	44
45	46	47	48	49	50	51	52	53
54	55	56	57	58	59	60	61	62
63	64	65	66	67	68	69	70	71
72	73	74	75	76	77	78	79	80

Figura 6.10
Aspecto de la aplicación
tras la creación de las casillas.

© FUNDAMENTAL

Function components frente a class components

El componente Casilla es un function component porque está creado basándose en una función, mientras que el componente Tablero es un class component. La diferencia entre ambos es que el function component no puede tener estado (state) y la forma de renderizarse es devolviendo con return el código JSX que genere el aspecto visual que se pretende de este.

Actividad propuesta 6.3



Para familiarizarse con ReactJS y Node, introduce el código anterior y verifica que tu navegador muestra un tablero como el de la figura 6.10.

6.11. Conectar el tablero con las casillas. State y props

En la aplicación, se necesita que la casilla tenga su propio estado. Por lo tanto, la casilla pasará de ser un componente funcional (function) a un class component que deriva de React.Component. La complejidad del componente aumenta y, por lo tanto, el cambio a class component es obligatorio. A continuación, se muestra el código nuevo para el componente Casilla:

```
class Casilla extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      value: props.value, // es el valor que se muestra al usuario
                        en el input
      original: props.original, // es el valor que tiene la casilla
                        en el sudoku resuelto
      fijo: props.fijo, //indicará si es un campo visible y no modificable
                        (readonly=true) o uno jugable (readonly=false)
    };
  }
  render() {
    return (
      <input type="text" className="casilla"
        value={this.state.value} // es el valor que se muestra
        readOnly={this.props.fijo} // si es fijo el campo no es
        modificable.
      />
    );
  }
}
```



```

        maxLength="1" //se limita a un dígito el número que se
        puede introducir.
      />
    );
  }
}

```

Ahora hay que modificar el componente Tablero. Hay que guardar el estado del juego y renderizar las casillas. El componente tiene que controlar lo siguiente:

- El valor de la casilla (número que tendría el sudoku resuelto).
- El valor que muestra al usuario para jugar.
- Se mostrarán solamente un 80% de las casillas, por eso se utiliza un número aleatorio entre 0 y 100. Si va de 0 a 80, se mostrará el valor y, en caso contrario, la casilla será jugable.
- Si la casilla es fija o jugable.
- Rellenar los valores anteriores al inicio de la clase (constructor). El constructor llamará a la función rellena para generar los valores del sudoku iniciales.
- El número de aciertos y fallos. De momento, no se controlan por código, pero serán necesarios más adelante. Se computarán en los métodos incFallo() e incAcertados().

El código, con sus respectivos comentarios, que cambia en el componente Tablero es el siguiente:

```

rellena(){
  generaSudoku();
  var i=0;
  var umbral=80;
  for (i=0;i<81;i++){
    /* se van recorriendo las casillas y se saca un número aleatorio entre 0 y 100
    Se establece un umbral de 80. Si el número aleatorio supera 80 la casilla
    estará vacía
    y deshabilitada (readOnly=false) para que juegue el usuario.
    En caso contrario estará readOnly (fija).
    */
    if (aleatorio(0,100) < umbral){
      this.state.casillasMostradas[i]=getSudoku(i); //muestra
      el valor original de la casilla
      this.state.fijo[i]=true; //coloca la casilla como readOn-
      ly
      this.state.acertados+=1; // se lleva un conteo de las ca-
      sillas acertadas. Si llega a 80 el sudoku estará resuelto
    }
    this.state.casillas[i]=getSudoku(i);
  }
}

```

```

constructor(props) {
  super(props);
  this.state = {
    casillas: Array(81).fill(null), // contiene el valor de la casilla
    casillasMostradas: Array(81).fill(null), //contiene si la casilla
    mostrada (inicialmente null).
    fijo: Array(81).fill(false), //contiene si la casilla es fija o
    jugable
    fallos: 0, //recuenta el número de fallos
    acertados: 0, //recuenta el número de aciertos. Si es igual a
    80 el juego ha terminado
    jugando: "jugando...", //almacena un literal. Cambiará cuando
    termine el juego
  };
  this.rellena(); //llama a la función rellena para rellenar las casillas
  y dejarlas fijas o jugables
}

incFallo(){ //este método incrementa en uno el número de fallos
  const fallos = this.state.fallos;
  this.setState({fallos:fallos+1});
}

incAcertados(){ //este método incrementa en uno el número de acertados
  const acertados=this.state.acertados;
  this.setState({acertados:acertados+1});
  if ((acertados+1) == 81){ //Si acierta 80 el juego ha terminado
    this.setState({jugando:"*****PARTIDA TERMINADA*****"});
  }
}

renderCasilla(i) {
  return (
    <Casilla
      value={this.state.casillasMostradas[i]}
      original={this.state.casillas[i]}
      fijo={this.state.fijo[i]}
      posicion={i}
    />
  );
}

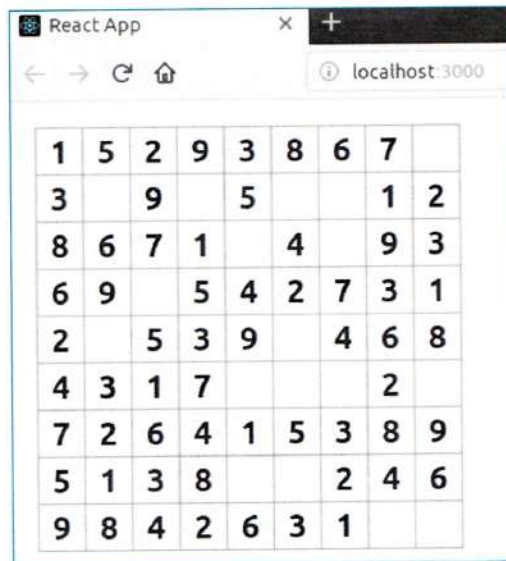
```



TOMA NOTA

El método render() no cambia, pero el componente Tablero sufre cambios para almacenar el estado del juego.

El aspecto de la aplicación en este momento será el que se muestra en la figura 6.11.



1	5	2	9	3	8	6	7	
3		9		5			1	2
8	6	7	1		4		9	3
6	9		5	4	2	7	3	1
2		5	3	9		4	6	8
4	3	1	7				2	
7	2	6	4	1	5	3	8	9
5	1	3	8			2	4	6
9	8	4	2	6	3	1		

Figura 6.11
Aspecto del juego
hasta el momento.



Actividad propuesta 6.4

Introduce el código expuesto en este apartado para comprobar cómo aparece en tu instalación Node un tablero parecido al de la figura 6.11.

6.12. Añadir interactividad. Los callbacks

Es el momento de añadir interactividad al juego. Habrá que hacer cambios en los componentes Tablero y Casilla para que reaccionen a las acciones del usuario.

6.12.1. Cambios en el componente Tablero

Los cambios realizados en el componente Tablero se exponen a continuación:

1. Se le añaden las casillas acertadas y los fallos. Obviamente, se parte de unas casillas acertadas. Se modifica el método `render()` añadiendo las líneas:

```
const status = 'Acertados: '+(this.state.acertados)+' Número de fallos: '
+ (this.state.fallos);
<div className="status">{status}</div>
```

2. Se indica si la partida está finalizada o se está jugando. Se modifica el método `render()` añadiendo la línea:

```
<div className="status">{this.state.jugando}</div>
```


3. Se añaden los callbacks. Se modifica el método `renderCasilla()` incluyendo en las props los métodos del componente `Tablero` `incFallo()` e `incAcertados()`. Se añaden las líneas siguientes al método `renderCasilla()`:

```
incFallo={() => this.incFallo()} //se envía como props la función incFallo
incAcertados={() => this.incAcertados()} //se envía como props la función
incAcertados
```

6.12.2. Cambios en el componente Casilla

Los cambios en el componente `Casilla` son los siguientes:

1. Al renderizarse la casilla, tiene que enlazar el evento `onBlur` (cuando el usuario abandona la casilla) con el método `handleBlur()`.

```
onBlur={this.handleBlur.bind(this)} // se enlaza el evento onBlur con el
método handleBlur
```

2. Se añade el método `handleBlur()` que controla el evento `onBlur`. Se incluye este método:

```
handleBlur(evt) {
  /*La función handleBlur es la que controla el evento onBlur (cuando
  el usuario abandona la casilla).
  Ese es el momento en el que el juego controla si el valor introdu-
  cido es correcto o no.*/
  if (evt.target.value !== ""){
    if (evt.target.value !== this.state.original){
      //Si el valor introducido no es igual al original el usuario
      ha fallado.
      this.props.incFallo();
    }else{
      //El usuario ha acertado. Se suma uno a los aciertos.
      if (this.state.value !== evt.target.value){
        this.props.incAcertados();
      }
    }
  }
}
```

6.12.3. Código final de la aplicación (archivo `index.js`)

A continuación, se muestra el código de la aplicación ya terminada:

```
import React from 'react';
import ReactDOM from 'react-dom';
```

```

import { aleatorio, getSudoku, generaSudoku } from './sudoku.js';
import './index.css';

class Casilla extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      value: props.value, // es el valor que se muestra al usuario en
                          // el input
      original: props.original, // es el valor que tiene la casilla
                          // en el sudoku resuelto
      fijo: props.fijo, //indicará si es un campo visible y no modifica-
                          // ble (readonly=true) o uno jugable (readonly=false)
    };
  }

  handleBlur(evt) {
    /*La función handleBlur es la que controla el evento onBlur
    (cuando el usuario abandona la casilla).
    Ese es el momento en el que el juego controla si el valor intro-
    ducido es correcto o no.*/
    if (evt.target.value !== ""){
      if (evt.target.value !== this.state.original){
        //Si el valor introducido no es igual al original el usuario
        ha fallado.
        this.props.incFallo();
      }else{
        //El usuario ha acertado. Se suma uno a los aciertos.
        if (this.state.value !== evt.target.value){
          this.props.incAcertados();
        }
      }
    }
  }

  render() {
    return (
      <input type="text" className="casilla"
        value={this.state.value} // es el valor que se muestra
        readOnly={this.props.fijo} // si es fijo el campo no es modificable.
        onBlur={this.handleBlur.bind(this)} // se enlaza el evento on-
        Blur con el método handleBlur
        maxLength="1" //se limita a un dígito el número que se puede
        introducir.
      />
    );
  }
}

```

```

class Tablero extends React.Component {

  rellena(){
    generaSudoku();
    var i=0;
    var umbral=80;
    for (i=0;i<81;i++){
      /* se van recorriendo las casillas y se saca un número aleatorio entre 0 y 100
      Se establece un umbral de 80. Si el número aleatorio supera 80 la casilla
      estará vacía
      y deshabilitada (readOnly=false) para que juegue el usuario.
      En caso contrario estará readOnly (fija).
      */
      if (aleatorio(0,100) < umbral){
        this.state.casillasMostradas[i]=getSudoku(i); //muestra el valor
          original de la casilla
        this.state.fijo[i]=true; //coloca la casilla como readOnly
        this.state.acertados+=1; // se lleva un conteo de las casillas acer-
          tadas. Si llega a 80 el sudoku estará resuelto
      }
      this.state.casillas[i]=getSudoku(i);
    }
  }

  constructor(props) {
    super(props);
    this.state = {
      casillas: Array(81).fill(null), // contiene el valor de la casilla
      casillasMostradas: Array(81).fill(null), //contiene si la casilla mostra-
        da (inicialmente null).
      fijo: Array(81).fill(false), //contiene si la casilla es fija o jugable
      fallos: 0, //recuenta el número de fallos
      acertados: 0, //recuenta el número de aciertos. Si es igual a 80 el juego
        ha terminado
      jugando: "jugando...", //almacena un literal. Cambiará cuando termine
        el juego
    };
    this.rellena(); //llama a la función rellena para rellenar las casillas y
      dejarlas fijas o jugables
  }

  incFallo(){ //este método incrementa en uno el número de fallos
    const fallos = this.state.fallos;
    this.setState({fallos:fallos+1});
  }

  incAcertados(){ //este método incrementa en uno el número de acertados
    const acertados=this.state.acertados;
    this.setState({acertados:acertados+1});
    if ((acertados+1) == 81){ //Si acierta 81 el juego ha terminado

```



```

    this.setState({jugando:"*****PARTIDA TERMINADA*****"});
  }
}

renderCasilla(i) {
  return (
    <Casilla
      value={this.state.casillasMostradas[i]}
      original={this.state.casillas[i]}
      fijo={this.state.fijo[i]}
      posicion={i}
      incFallo={() => this.incFallo()} //se envía como props la
      función incFallo
      incAcertados={() => this.incAcertados()} //se envía como
      props la función incAcertados
    />
  );
}

render() {
  const status = 'Acertados: '+(this.state.acertados)+' Número de
fallos: ' + (this.state.fallos);
  return (
    <div>
      <div className="status">{status}</div>
      <div className="tablero-fila">
        {this.renderCasilla(0)}
        {this.renderCasilla(1)}
        {this.renderCasilla(2)}
        {this.renderCasilla(3)}
        {this.renderCasilla(4)}
        {this.renderCasilla(5)}
        {this.renderCasilla(6)}
        {this.renderCasilla(7)}
        {this.renderCasilla(8)}
      </div>
      <div className="tablero-fila">
        {this.renderCasilla(9)}
        {this.renderCasilla(10)}
        {this.renderCasilla(11)}
        {this.renderCasilla(12)}
        {this.renderCasilla(13)}
        {this.renderCasilla(14)}
        {this.renderCasilla(15)}
        {this.renderCasilla(16)}
        {this.renderCasilla(17)}
      </div>
      <div className="tablero-fila">

```

```
{this.renderCasilla(18)}
{this.renderCasilla(19)}
{this.renderCasilla(20)}
{this.renderCasilla(21)}
{this.renderCasilla(22)}
{this.renderCasilla(23)}
{this.renderCasilla(24)}
{this.renderCasilla(25)}
{this.renderCasilla(26)}
</div>
<div className="tablero-fila">
  {this.renderCasilla(27)}
  {this.renderCasilla(28)}
  {this.renderCasilla(29)}
  {this.renderCasilla(30)}
  {this.renderCasilla(31)}
  {this.renderCasilla(32)}
  {this.renderCasilla(33)}
  {this.renderCasilla(34)}
  {this.renderCasilla(35)}
</div>
<div className="tablero-fila">
  {this.renderCasilla(36)}
  {this.renderCasilla(37)}
  {this.renderCasilla(38)}
  {this.renderCasilla(39)}
  {this.renderCasilla(40)}
  {this.renderCasilla(41)}
  {this.renderCasilla(42)}
  {this.renderCasilla(43)}
  {this.renderCasilla(44)}
</div>
<div className="tablero-fila">
  {this.renderCasilla(45)}
  {this.renderCasilla(46)}
  {this.renderCasilla(47)}
  {this.renderCasilla(48)}
  {this.renderCasilla(49)}
  {this.renderCasilla(50)}
  {this.renderCasilla(51)}
  {this.renderCasilla(52)}
  {this.renderCasilla(53)}
</div>
<div className="tablero-fila">
  {this.renderCasilla(54)}
  {this.renderCasilla(55)}
  {this.renderCasilla(56)}
  {this.renderCasilla(57)}
```

```

        {this.renderCasilla(58)}
        {this.renderCasilla(59)}
        {this.renderCasilla(60)}
        {this.renderCasilla(61)}
        {this.renderCasilla(62)}
    </div>
    <div className="tablero-fila">
        {this.renderCasilla(63)}
        {this.renderCasilla(64)}
        {this.renderCasilla(65)}
        {this.renderCasilla(66)}
        {this.renderCasilla(67)}
        {this.renderCasilla(68)}
        {this.renderCasilla(69)}
        {this.renderCasilla(70)}
        {this.renderCasilla(71)}
    </div>
    <div className="tablero-fila">
        {this.renderCasilla(72)}
        {this.renderCasilla(73)}
        {this.renderCasilla(74)}
        {this.renderCasilla(75)}
        {this.renderCasilla(76)}
        {this.renderCasilla(77)}
        {this.renderCasilla(78)}
        {this.renderCasilla(79)}
        {this.renderCasilla(80)}
    </div>
    <div className="status">{this.state.jugando}</div>
</div>

    );
}
}

class Juego extends React.Component {
    render() {
        return (
            <div className="juego">
                <div className="juego-tablero">
                    <Tablero />
                </div>
                <div className="juego-info">
                </div>
            </div>
        );
    }
}

// =====

```



```
ReactDOM.render(
  <Juego />,
  document.getElementById('root')
);
```

El aspecto de la aplicación ahora es el que se observa en la figura 6.12.

Acertados: 65 Número de fallos: 0

8	6	7	1	4	2	9	3	5
1	5	2	9	8	3	7	4	6
3	4	9	6	7		1	2	8
	9	8	5	2	4	3	1	7
4		1	7	6		2	5	
2	7			1	9		8	4
7		6			1	8	9	3
5	1	3	8	9	7	4	6	2
9	8				6			1

jugando...

Figura 6.12

Aspecto final de la aplicación.

6.13. Prácticas guiadas

A continuación, se muestran dos prácticas que utilizan dos de las técnicas más empleadas en React, las props y los callbacks.

6.13.1. Conversor de euros a dólares

Se quiere realizar un conversor de euros a dólares. Supóngase que el cambio actual es 0,8 euros = 1 dólares.

Se quiere que tenga el aspecto que se presenta en la figura 6.13.

0 euros

0 dólares

Figura 6.13

Aspecto del ejercicio para crear un conversor de euros en dólares.

Cuando se pulse el botón +, se le sumará a la cantidad en euros 1 euro y se convertirá a dólares, mientras que, cuando se pulse el botón -, se le restará 1 euro.

Para la solución a este ejercicio, se ha creado un componente principal llamado *App* que tendrá en su state los valores de dólares y euros.

Además, se crean dos componentes subordinados (Aumentar y Disminuir) que, en este caso, no son clases, sino funciones, ya que tienen una funcionalidad simple. A dichos componentes se les pasan por props las funciones aumentar() y disminuir() del componente padre App. En el evento onClick de dichos botones, se ejecutarán las funciones anteriormente pasadas vía props.

Para simplificar, se han integrado todos los componentes en un único archivo. Normalmente, cada componente se ubica en un archivo separado.

```
import React, { Component } from 'react';
import logo from './logo.svg';
import './App.css';

class App extends Component {
  constructor(props) {
    super(props);
    this.state = {
      dolares: 0,
      euros: 0,
    };
  }

  aumentar(){
    var auxeuro = this.state.euros+1;
    this.setState({euros:auxeuro});
    this.setState({dolares:auxeuro*1.083});
  }
  disminuir(){
    var auxeuro = this.state.euros-1;
    this.setState({euros:auxeuro});
    this.setState({dolares:auxeuro*1.083});
  }

  render() {
    return (
      <div className="App">
        <br />
        {this.state.euros} euros <Aumentar aumentar={() => this.aumentar()} /><Disminuir disminuir={() => this.disminuir()} /><-br />
        {this.state.dolares} dólares<br />
      </div>
    );
  }
}

function Aumentar(props) {
  return (
    <button className="aumentar" onClick={props.aumentar}>
      +
    </button>
  );
}
```

```

        </button>
      );
    }
    function Disminuir(props) {
      return (
        <button className="aumentar" onClick={props.disminuir}>
          -
        </button>
      );
    }
  }

  export default App;

```

Actividad propuesta 6.5



Partiendo del código expuesto en este apartado, añade una divisa más: el yen japonés. De esta manera, el programa mostrará el cambio en dólares y yenes.

6.13.2. Conversor de dólares a euros, y viceversa

Se quiere realizar un conversor de euros a dólares, y viceversa. El cambio actual es 1 euro = 0,80 dólares.

Se quiere que tenga el mismo aspecto que el que se muestra en la figura 6.14.

Figura 6.14

Aspecto del ejercicio para crear un conversor de dólares en euros, y viceversa.

Cuando se introduzca una cifra en dólares, se mostrarán en euros, y viceversa.

Como en el ejercicio anterior, se han unificado todos los componentes en un único archivo, pero cada componente suele ubicarse en un archivo separado.

En este caso, la aplicación es parecida a la del apartado 6.13.1, pero, en el componente subordinado Moneda, se enlaza (bind) el evento onChange con la función handleChange(), la cual hace una llamada al método convertir enviado vía props. Este método o función tiene dos parámetros. El primero será “dolar” o “euro” para conocer el campo que ha sido modificado y el segundo, el valor actual de dicho campo. Dicho valor se recoge en el atributo evt.target.value de la función handleChange(). El objeto evt recoge el evento producido; el objeto target, el objeto que ha producido dicho evento, y, en value, se recogerá el value (número) del input utilizado.


```

import React, { Component } from 'react';
import logo from './logo.svg';
import './App.css';

class App extends Component {
  constructor(props) {
    super(props);
    this.state = {
      dolares: 0,
      euros: 0,
    };
  }

  convertir(moneda, valor){
    if (moneda=="dolar"){
      this.setState({dolares:valor});
      this.setState({euros:valor/0.8});
    }
    if (moneda=="euro"){
      this.setState({dolares:valor*0.8});
      this.setState({euros:valor});
    }
  }

  render() {
    return (
      <div className="App">
        <Moneda moneda="dolar" value={this.state.dolares} convertir={{(-
moneda,valor) => this.convertir(moneda,valor)}}/>
        Dólares ---- Euros
        <Moneda moneda="euro" value={this.state.euros} convertir={{(mo-
neda,valor) => this.convertir(moneda,valor)}}/>

      </div>
    );
  }
}

class Moneda extends Component{
  handleChange(evt) {
    this.props.convertir(this.props.moneda,evt.target.value);
  }
  render(){
    return(
      <input type="number"
        value = {this.props.value}
        onChange={this.handleChange.bind(this)}
      />
    );
  }
}

export default App;

```

Actividad propuesta 6.6



Introduce el código expuesto en este apartado para comprobar cómo funciona el conversor en tu instalación Node. Una vez verificado, añade una divisa más: el yen japonés.

Resumen

- Las aplicaciones React están formadas por componentes.
- ReactJS utiliza una sintaxis parecida al XML llamada *JSX*.
- Lo normal en ReactJS es programar en *JSX*, pero no es absolutamente necesario.
- Yarn, al igual que NPM, es una herramienta para gestionar paquetes de aplicaciones Node.js.
- La utilidad *create-react-app* permite crear el esqueleto de una aplicación básica.
- El método *render()* en un componente React es el encargado de mostrar el componente en la interfaz web.
- *JSX* es una extensión de JavaScript.
- Las interfaces de usuario se compondrán de componentes que son independientes unos de otros.
- Los componentes son piezas reutilizables, como las piezas de LEGO.
- Un componente devolverá una serie de elementos que aparecerán en la ventana del navegador.
- En React, se pueden definir diferentes clases o tipos de componentes.
- Los componentes realizados por el programador deben tener la primera letra en mayúsculas obligatoriamente.
- Los componentes en React se pueden clasificar dependiendo de su uso en presentacionales o contenedores.
- El *state* representará el estado del componente y las *props* (o propiedades) son los parámetros que le pasa un componente padre a su componente subordinado o hijo.
- Solamente los *class components* pueden tener estado local (*state*).
- React utiliza un objeto que mantiene un DOM virtual, el cual es una representación ligera del DOM del navegador.
- React tiene un algoritmo que detecta los nodos que cambian y se encarga de re-renderizarlos.
- El envío de vuelta de información se denomina *callback* y es una función que se pasa al componente subordinado (hijo) para que la ejecute con sus datos.

Ejercicios propuestos



1. Bootstrap da mucha más versatilidad, calidad y definición a una aplicación web. Para incluir bootstrap en el proyecto, instala las siguientes librerías: bootstrap, reactstrap y react-popper.

Para más información sobre reactstrap, consulta <https://reactstrap.github.io> y, para obtener más información de cómo utilizar Navbar de reactstrap, accede a <https://reactstrap.github.io/components/navbar>.

2. Incluye una barra de navegación en el sudoku e importa librerías. Para ello, añade las siguientes líneas a tu archivo index.js:

```
import { Navbar,NavbarBrand } from 'reactstrap';
import 'bootstrap/dist/css/bootstrap.min.css';
```

Las líneas anteriores son para importar los componentes Navbar y NavbarBrand de la librería reactstrap.

3. Crea un componente Header que renderice el siguiente código:

```
<Navbar dark color="primary">
  <div className="container">
    <NavbarBrand href="/">SUDOKU en ReactJS</NavbarBrand>
  </div>
</Navbar>
```

Llama al componente desde el componente Juego para que aparezca en la parte superior de la pantalla la barra de navegación creada.

4. Instala las siguientes librerías como administrador: font-awesome y bootstrap-social.
5. Teniendo el siguiente código dentro de un componente React:

```
constructor(){
  super();
  this.state = {
    saludo: "Esta aplicación saluda en varios idiomas"
  };
  this.mensajeIngles = this.mensajeIngles.bind(this);
}
mensajeIngles(){
  this.setState({
    saludo: "Hello"
  });
}
```

¿Por qué se tiene que incluir en el constructor la siguiente línea?

```
this.mensajeIngles = this.mensajeIngles.bind(this);
```


6. Crea el componente Footer (archivo FooterComponent.js) con el siguiente código:

```
import React from 'react';
import 'font-awesome/css/font-awesome.css';
import 'bootstrap-social/bootstrap-social.css';

function Footer(props) {
  return(
    <div className="footer">
      <div className="container">
        <div className="row justify-content-center">
          <div className="col-4 offset-1 col-sm-2">
            <h5>Links</h5>
            <ul className="list-unstyled">
              <li><a href="#">Home</a></li>
              <li><a href="#">About</a></li>
              <li><a href="contacto.html">Contacto</a></li>
            </ul>
          </div>
          <div className="col-7 col-sm-5">
            <h5>Dirección de mi aplicación Sudoku</h5>
            <address>
              Dirección completa<br />
              de mi empresa sudoku<br />
              ANDALUCÍA<br />
              <i className="fa fa-phone fa-lg"></i>: +123456789<br />
              <i className="fa fa-fax fa-lg"></i>: +123456789<br />
              <i className="fa fa-envelope fa-lg"></i>: <a href="mailto:jcmoreno@sintesis.com">
                jcmoreno@sintesis.com</a>
            </address>
          </div>
          <div className="col-12 col-sm-4 align-self-center">
            <div className="text-center">
              <a className="btn btn-social-icon btn-google" href="http://google.com/+"><i className="fa fa-google-plus"></i></a>
              <a className="btn btn-social-icon btn-facebook" href="http://www.facebook.com/profile.php?id="><i className="fa fa-facebook"></i></a>
            </div>
          </div>
        </div>
      </div>
    </div>
  );
}
```

```

      <a className="btn btn-social-icon btn-linkedin" href="http://www.linkedin.com/in/"><i
        className="fa fa-linkedin"></i></a>
      <a className="btn btn-social-icon btn-twitter" href="http://twitter.com/"><i classNa
        me="fa fa-twitter"></i></a>
      <a className="btn btn-social-icon btn-google" href="http://youtube.com/"><i className="fa
        fa-youtube"></i></a>
      <a className="btn btn-social-icon" href="mailto:"><i className="fa fa-envelope-o"></
        i></a>
    </div>
  </div>

  </div>
  <div className="row justify-content-center">
    <div className="col-auto">
      <p>© Copyright 2018 Sudoku company</p>
    </div>
  </div>
</div>
</div>
)
}

export default Footer;

```

7. Importa el componente Footer e inclúyelo en el proyecto. Fichero index.js:

```

import Footer from './FooterComponent';
<Footer />
...

```

8. Modifica el CSS del proyecto añadiendo las siguientes clases:

```

.footer{
  background-color: #A9CCE3;
  margin:0px auto;
  padding: 20px 0px 20px 0px;
}
address{
  font-size:80%;
  margin:0px;
  color:#0f0f0f;
}

```


9. Centra el cuadro de juego del sudoku para que quede con la apariencia de la figura 6.15.



Figura 6.15

Aplicación sudoku finalizada con Header y Footer.

10. Realiza una página web que salude en cuatro idiomas (figura 6.16).

Esta aplicación saluda en varios idiomas

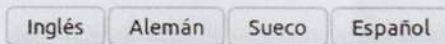


Figura 6.16

Resultado del ejercicio.

A continuación, se proporciona una parte del componente Saluda, finaliza el componente y también la página web.

```
class Saluda extends React.Component {

  constructor(){
    super();
    this.state = {
      saludo: "Esta aplicación saluda en varios idiomas"
    };
  }
  render() {
    return (
      <div>
        <h3> {this.state.saludo}</h3>
        <button onClick={this.mensajeIngles}>Inglés</but-
        ton>
      </div>
    );
  }
}
```



```

        <button onClick={this.mensajeAleman}>Alemán</button>
        <button onClick={this.mensajeSueco}>Sueco</button>
        <button onClick={this.mensajeEspanol}>Español</button>
      </div>
    )
  }
}

```

ACTIVIDADES DE AUTOEVALUACIÓN

- ¿Cuál de las siguientes props es correcta?
 - ☐ a) `incFallo={this.incFallo()}`
 - ☐ b) `incFallo={() => this.incFallo()}`
 - ☐ c) `incFallo={() = this.incFallo()}`
- Para enlazar el evento `onBlur` con el método `handleBlur()`, ¿qué sentencia se utiliza?
 - ☐ a) `onBlur={this.handleBlur.bind(this)}`
 - ☐ b) `onBlur={bind(this.handleBlur)}`
 - ☐ c) `bind(onBlur,this.handleBlur())`
- ¿Cuál de las siguientes órdenes es la correcta?
 - ☐ a) `<Casilla posicion={i}> </Casilla>`
 - ☐ b) `<Casilla posicion=this.props.i> </Casilla>`
 - ☐ c) `<Casilla posicion={i} />`
- ¿Qué sentencia se utiliza para importar funciones de un archivo JavaScript del proyecto?
 - ☐ a) `import { aleatorio(),getSudoku(), generaSudoku() } from './sudoku.js';`
 - ☐ b) `import { aleatorio,getSudoku, generaSudoku } from './sudoku.js';`
 - ☐ c) `import { #aleatorio, #getSudoku, #generaSudoku } from './sudoku.js';`
- En un constructor, siempre tiene que existir la llamada:
 - ☐ a) `super(props);`
 - ☐ b) `super(this);`
 - ☐ c) `parent(props);`

6. ¿Cuál de las siguientes afirmaciones sobre Flux y Redux es correcta?
- ☐ a) Flux y Redux son librerías.
 - ☐ b) Flux es un patrón de diseño, mientras que Redux es una librería.
 - ☐ c) Flux y Redux son frameworks.
7. ¿Cuál es la afirmación verdadera sobre JSX?
- ☐ a) JSX es una extensión de JavaScript.
 - ☐ b) JSX es una extensión de HTML.
 - ☐ c) JSX es una extensión de XML.
8. ¿Qué es ReactJS?
- ☐ a) Technology Program Agnostic.
 - ☐ b) Technology Component Based Agnostic.
 - ☐ c) Technology Stack Agnostic.
9. ¿En qué fichero está la línea `<div id="root"></div>`?
- ☐ a) `public/index.html`.
 - ☐ b) `src/App.js`.
 - ☐ c) `src/index.js`.
10. ¿Cuál de las siguientes sentencias es la correcta?
- ☐ a) `class Tablero extends Component {...}`
 - ☐ b) `class Tablero extends React.Component {...}`
 - ☐ c) `class Tablero extends React.DOM.Component {...}`

SOLUCIONES:

1. ☐ a ☒ b ☐ c
2. ☒ a ☐ b ☐ c
3. ☐ a ☐ b ☒ c
4. ☐ a ☒ b ☐ c

5. ☒ a ☐ b ☐ c
6. ☐ a ☒ b ☐ c
7. ☒ a ☐ b ☐ c
8. ☐ a ☐ b ☒ c

9. ☒ a ☐ b ☐ c
10. ☐ a ☒ b ☐ c